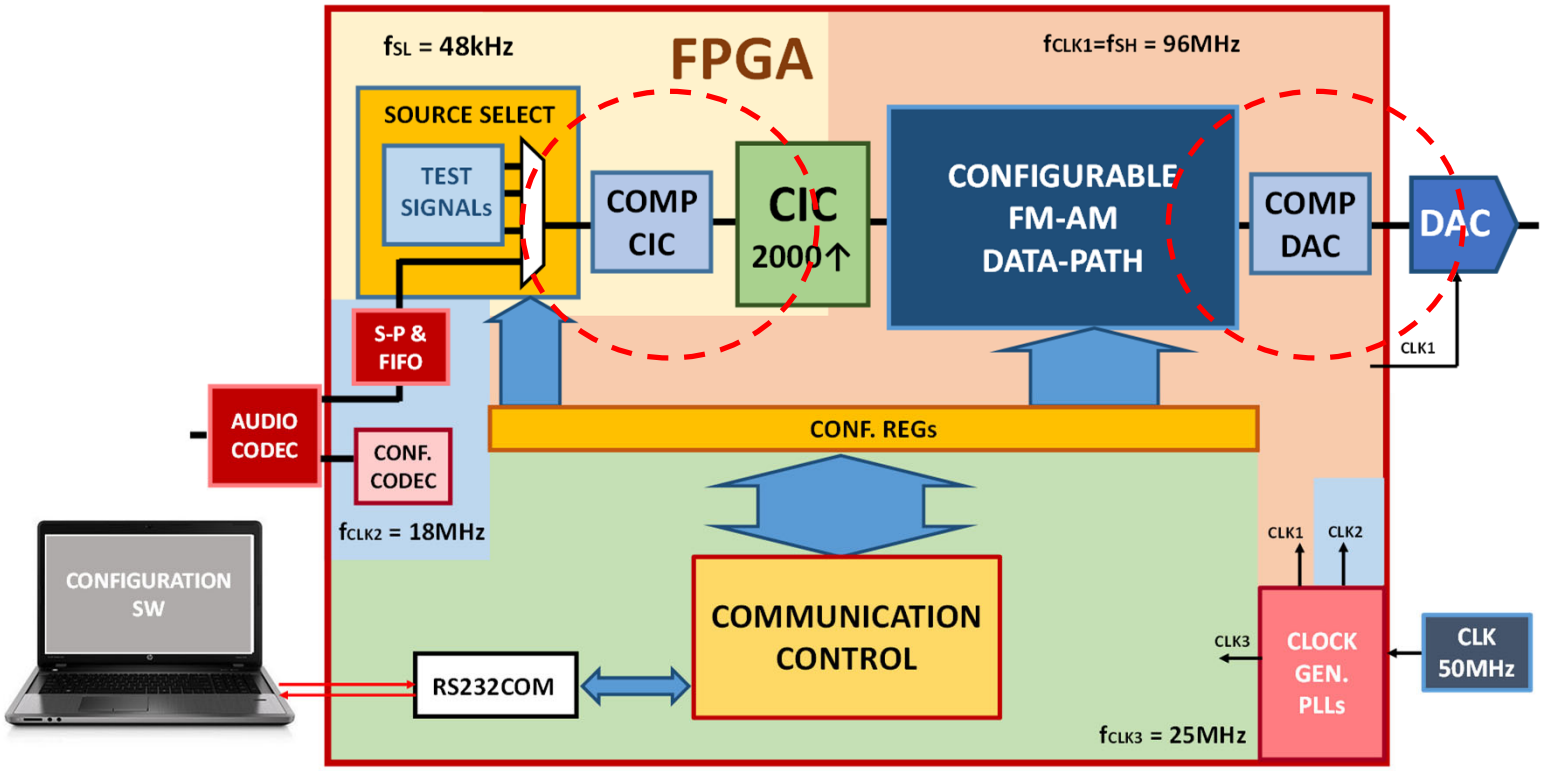
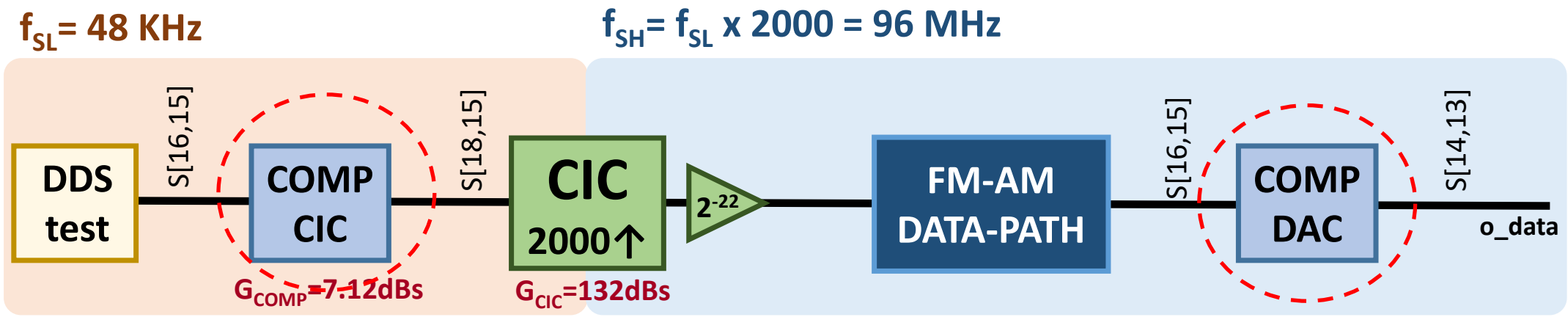
P4 Modulador configurable AM/FM



P1: Sintetizador de frecuencias (DDS)
P2: Ruta de datos AM/FM configurable
P3: Filtro interpolador CIC
P4: Filtros compensadores CIC y DAC

P5: Comunicación con PC, control
P6: Completar sistema

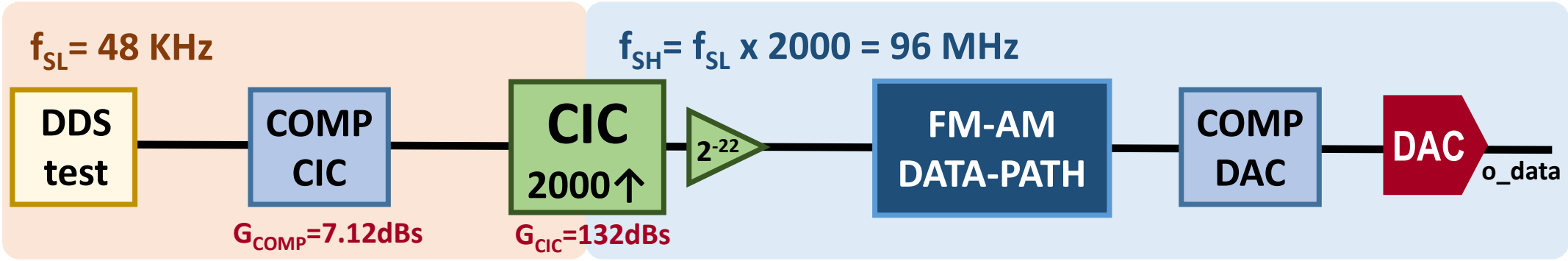
P4 Ruta de datos del modulador AM-FM



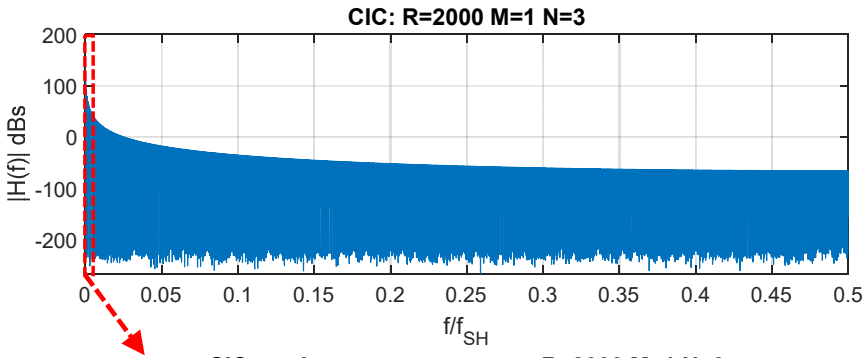
P4_1: Filtro compensador de la respuesta en frecuencia del CIC

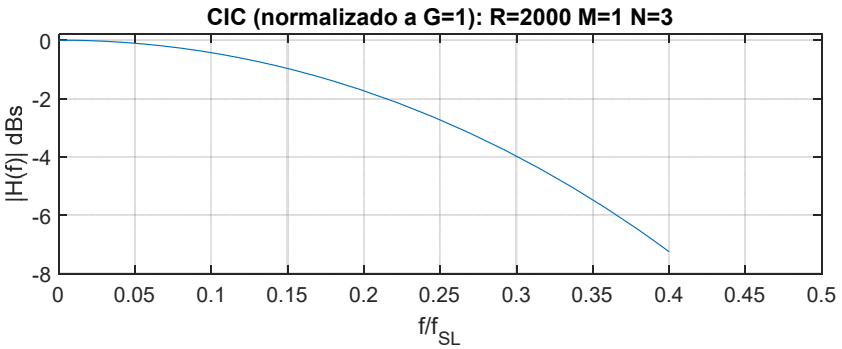
P4_2: Filtro compensador de la respuesta en frecuencia del DAC

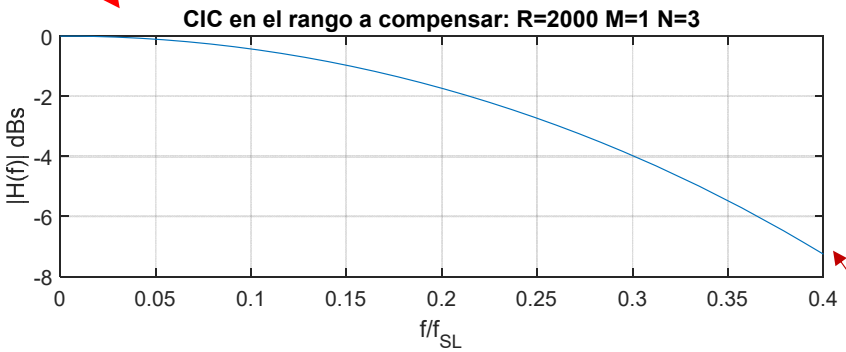
P4.2 Ruta de datos: filtro compensador del CIC

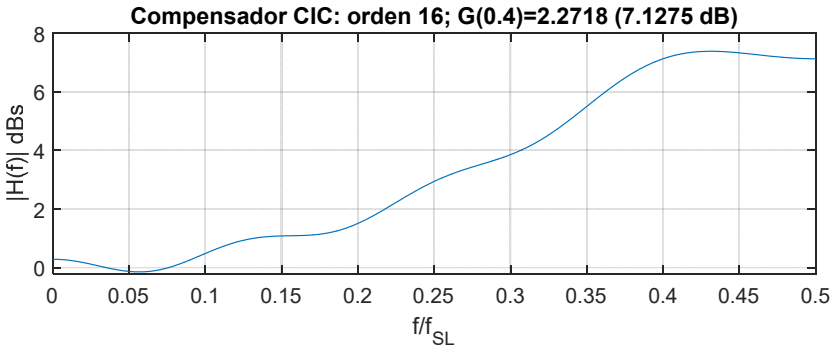


P4.1 ¿Por qué necesitamos un filtro compensador?



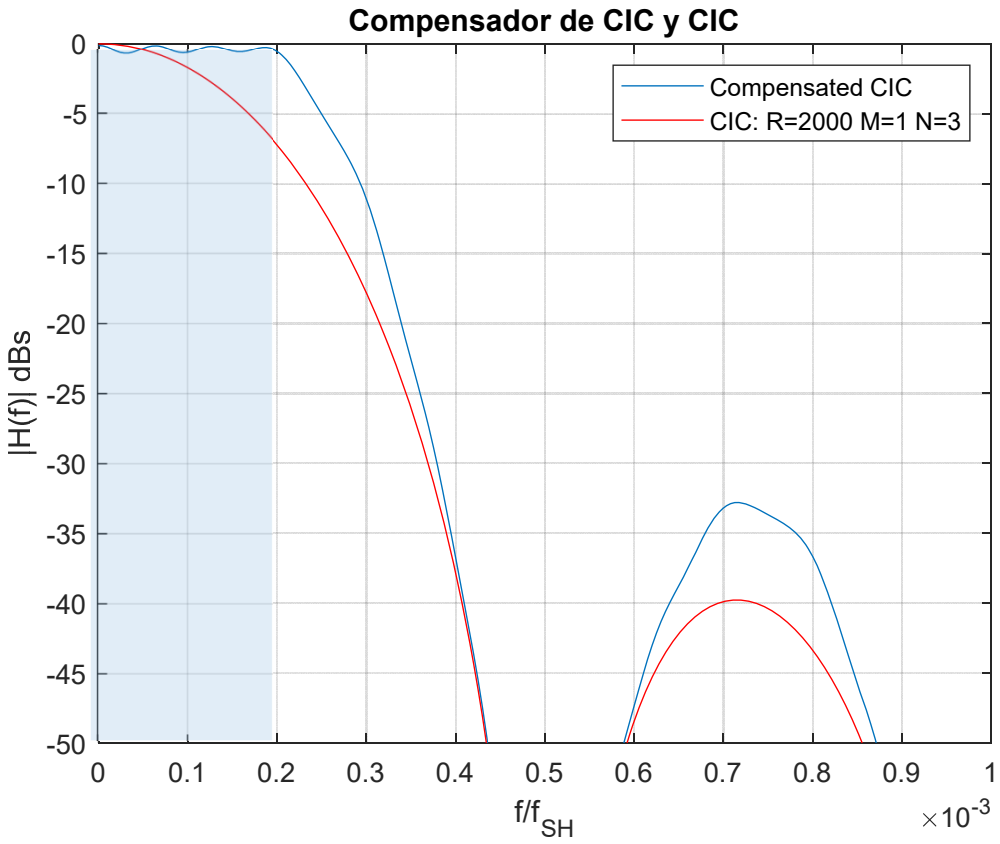






f_{SL}=43 kHz f=18.4 kHz → f/f_{SL} = 0.4 |H(f)|= -7.25dBs

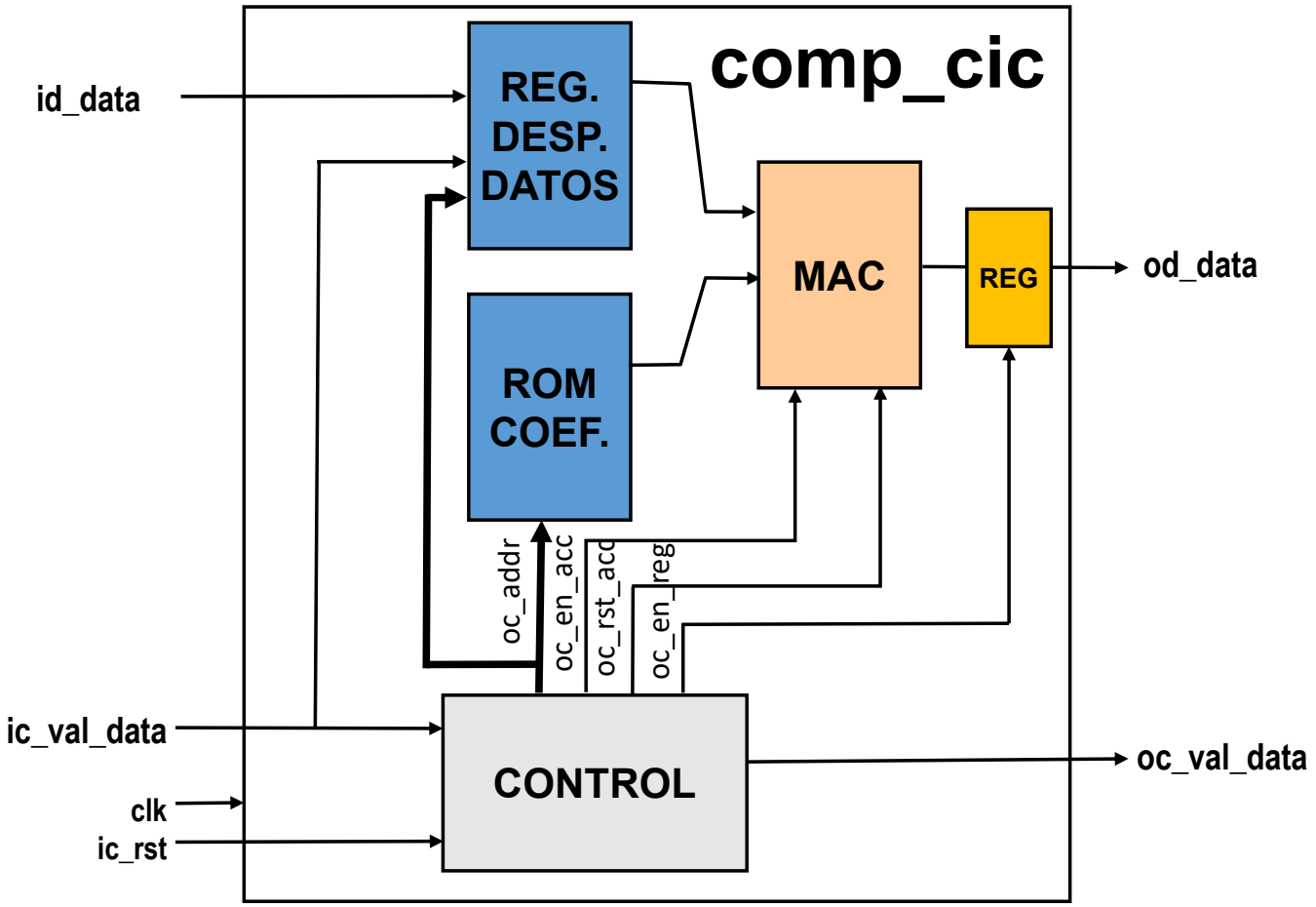
P4.1 Filtro FIR Compensador del CIC



f_{SH}=96 MHz f=15 kHz → f/f_{SH} = 0.19e-3

|H(f)|= -0.29 dBs

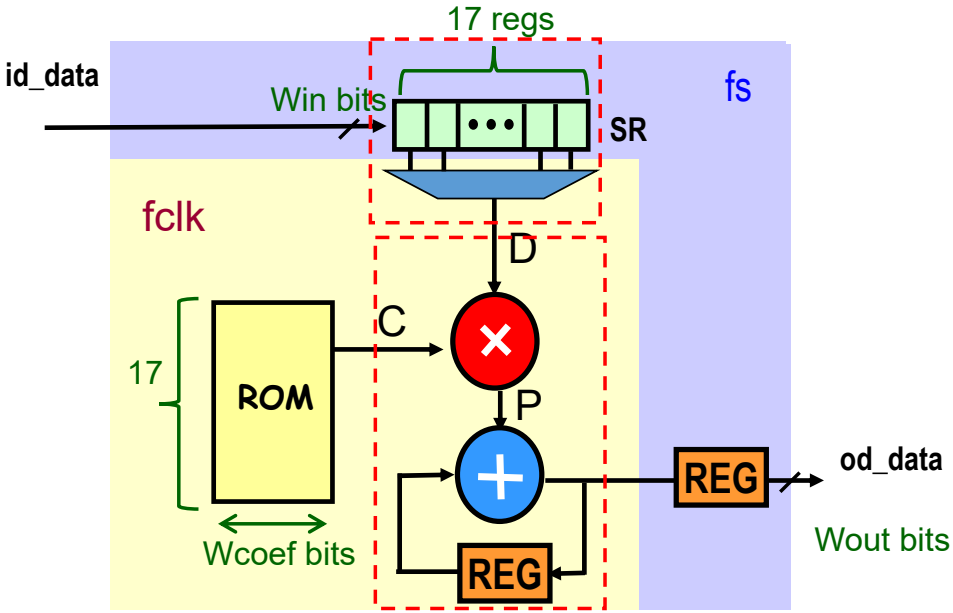
P4.1 Filtro FIR secuencial Compensador del CIC

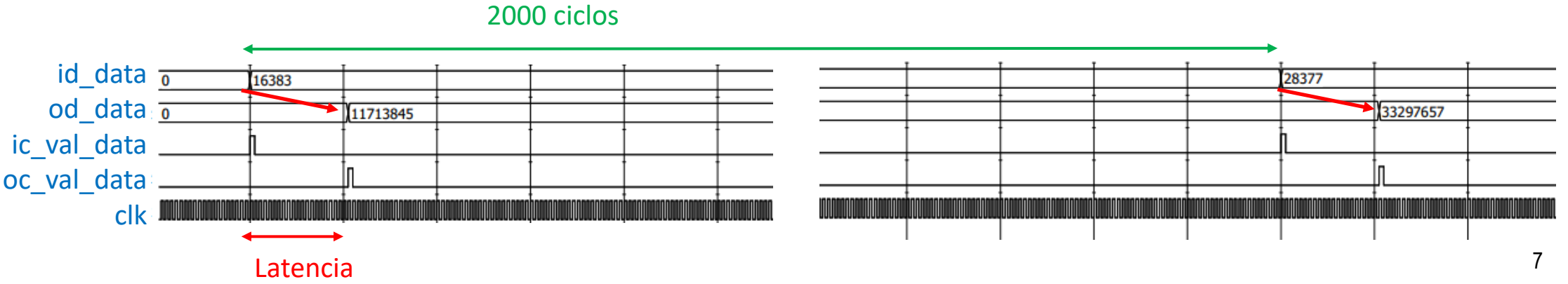


P4.1 Filtro FIR secuencial Compensador del CIC

Especificaciones
Número de etapas: N=17
Coeficientes h_comp simétricos
Tamaño de la entrada: Win = 16 bits
Tamaño de la salida Wout = 18 bits
FPGA Cyclone IV EP4CE115F29C7

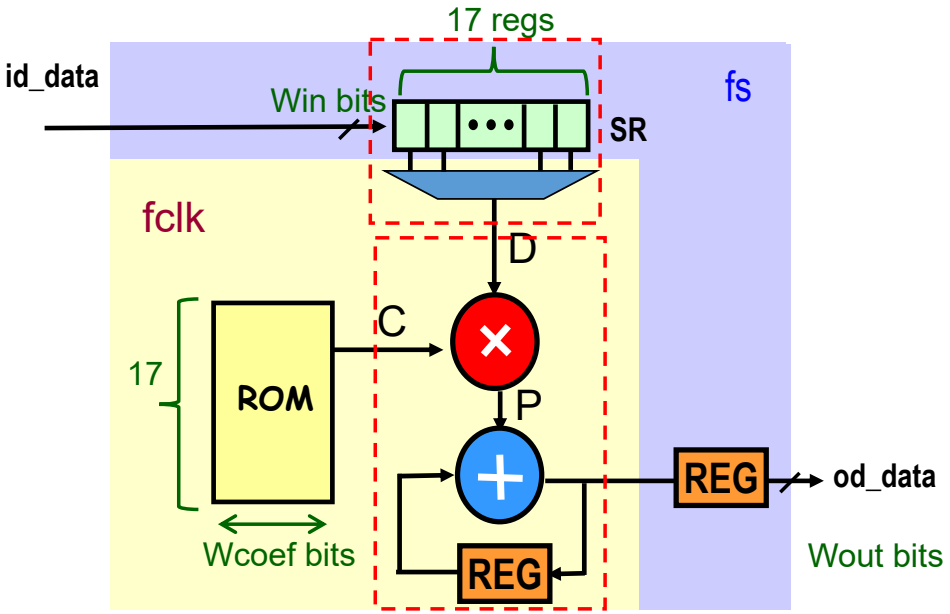
El ancho de banda de la señal de entrada será < 20 kHz
fclk = 96 MHz
fs = 48 kHz





P4.1 Filtro FIR: cuantificación interna

Id_data S[16,15]
Coeficiente S[Wcoef, Fcoef]



id_data→ $X=Xe2^{-15}$ (16 bits)
Coefs →$H=He2^{-Fcoef}$ (Wcoef bits)
$P_j=X_i \cdot H_j$ (16+Wcoef bits)

$$Ng=\log_2(Ge),\ Ge=G_{lin}*2^{Fcoef}$$

Ng : Crecimiento de la cuantificación del filtro

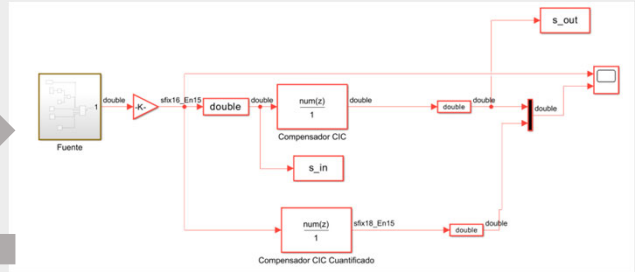
P4.1 Método de verificación comp_cic_pc

Ficheros de P4_1

.\mat\sim_comp_cic_model.m
.\mat\comp_cic_model.slx
.\src\comp_cic_reg_mux.sv, comp_ cic_rom.sv, cic_rom.sv, rom_coefs_comp_cic.txt, comp_ cic_mult_acc.sv, comp_cic_control.sv y comp_cic_pc.sv
.\tsb\comp_cic_reg_mux_tsb.sv, comp_cic_rom_tsb.sv, comp_cic_mut_acc_tsb.sv, comp_cic_control.sv y comp_cic_tsb.sv → Bancos de pruebas
.\tsb\comp_cic_tsb_pkg.sv → Parámetros
.\sim\iof\id_comp_cic.txt, od_comp_cic.txt → Entrada y salida banco de pruebas

Matlab

sim_comp_cic_model.m
- Configura Win, Wout, Wcoef, Num_coef
- Lanza comp_cic_model.slx
- Recoge datos
- Escribe ficheros de test



comp_cic_model.slx

id_comp_cic.txt

comp_cic_tsb_pkg.sv

rom_coefs_comp_cic.txt

od_comp_cic.txt

comp_cic_tsb

id_data
ic_ rst
ic_ val_data
clk

comp_cic_pc

od_data
oc_val_data

CE
CE

o_data_F
o_data_M

Error check

if (*_M ≠ *_F)
error_cnt = error_cnt+1

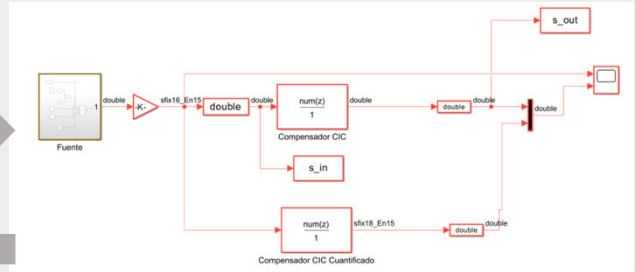
P4.1 Método de verificación comp_cic

Ficheros de P4_1

.\mat\sim_comp_cic_model.m
.\mat\comp_cic_model.slx
.\src\comp_cic_reg_mux.sv, comp_ cic_rom.sv, cic_rom.sv, rom_coefs_comp_cic.txt, comp_ cic_mult_acc.sv, comp_cic_control.sv y comp_cic_pc.sv, comp_cic.sv
.\tsb\comp_cic_reg_mux_tsb.sv, comp_cic_rom_tsb.sv, comp_cic_mut_acc_tsb.sv, comp_cic_control.sv y comp_cic_tsb.sv → Bancos de pruebas
.\tsb\comp_cic_tsb_pkg.sv → Parámetros
.\sim\iof\id_comp_cic.txt, od_comp_cic.txt → Entrada y salida banco de pruebas

Matlab

sim_comp_cic_model.m
- Configura Win, Wout, Wcoef, Num_coef
- Lanza comp_cic_model.slx
- Recoge datos
- Escribe ficheros de test



comp_cic_model.slx

id_comp_cic.txt

comp_cic_tsb_pkg.sv

rom_coefs_comp_cic.txt

od_comp_cic.txt

comp_cic_tsb

id_data
ic_ rst
ic_ val_data
clk

comp_cic

od_data
oc_val_data

CE
CE

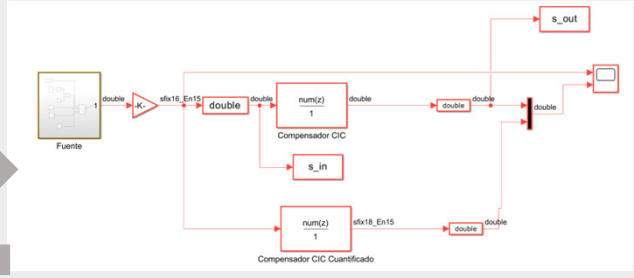
o_data_F
o_data_M

Error check

if (*_M ≠ *_F)
error_cnt = error_cnt+1

P4.1 Método de verificación comp_cic

sim_comp_cic_model.m
- Configura Win, Wout, Wcoef, Num_coef
- Lanza comp_cic_model.slx
- Recoge datos
- Escribe ficheros de test



comp_cic_model.slx

Matlab

id_comp_cic.txt
- Datos de entrada
- Señal: s_in

od_comp_cic_pc.txt
- Datos de salida
- Señal: salida precisión finita

comp_cic_tsb_pkg.sv

parameter =
- test_case
- num_data_in
- num_data_out
- Win
- Ng
- R
- Wcoef
- Wout
- Num_coef
- full_precision

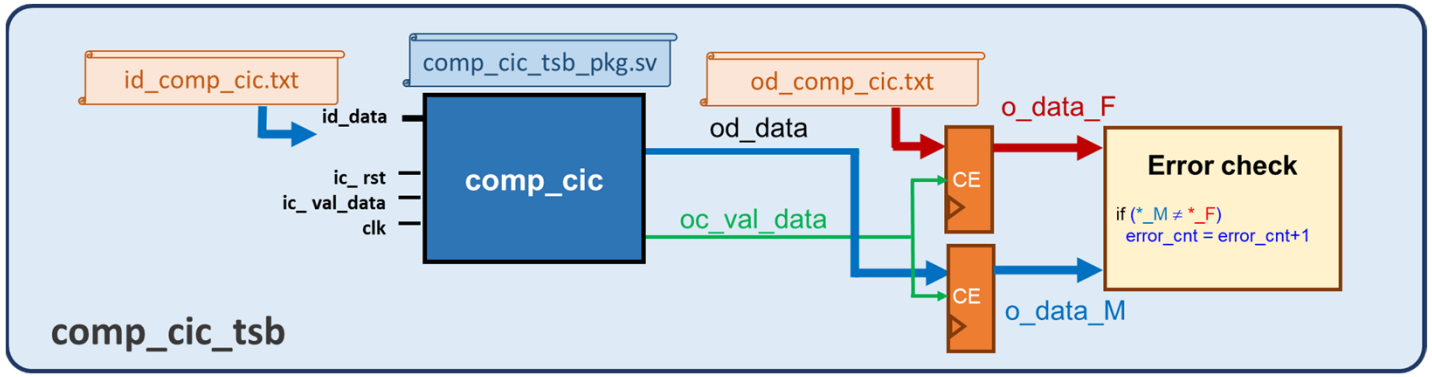
rom_coefs_comp_cic.txt
- Coeficientes del filtro

Ficheros de P4_1

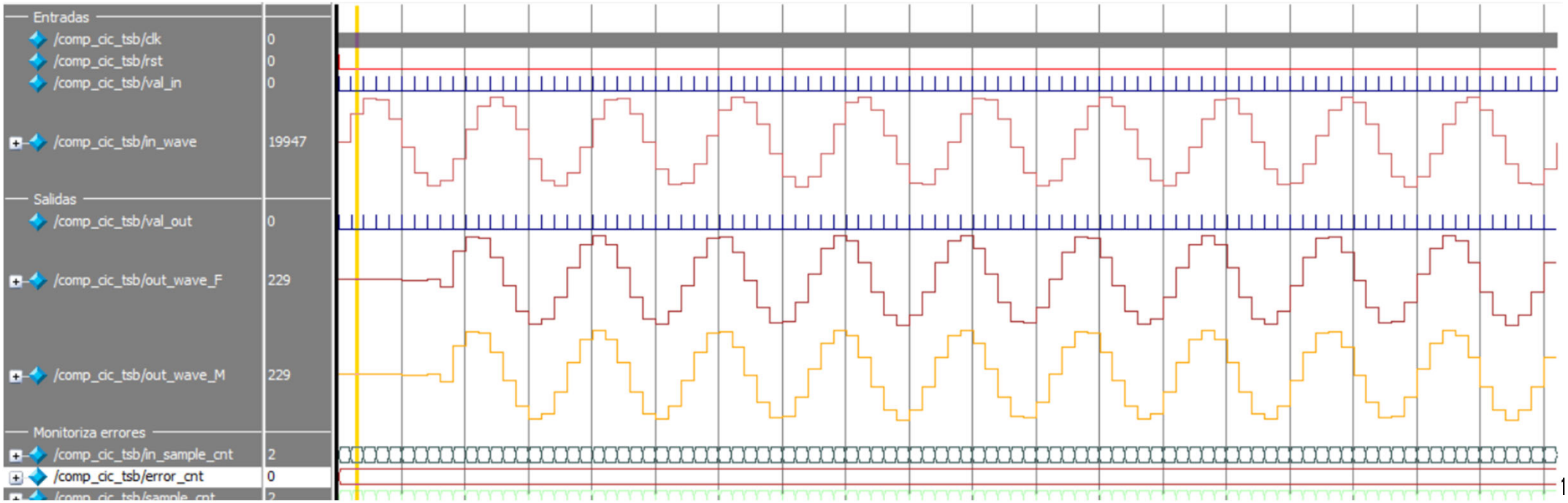
.\mat\sim_comp_cic_model.m
.\mat\comp_cic_model.slx
.\src\comp_cic_reg_mux.sv, comp_ cic_rom.sv, comp_ cic_mult_acc.sv, comp_cic_control.sv, comp_cic_pc.sv y comp_cic.sv
.\tsb\comp_cic_reg_mux_tsb.sv, comp_cic_rom_tsb.sv, comp_cic_mut_acc_tsb.sv, comp_cic_control.sv y comp_ cic_tsb.sv → Bancos de pruebas
.\tsb\comp_cic_tsb_pkg.sv → Parámetros
.\sim\iof\id_comp_cic.txt, od_comp_cic.txt → Entrada y salida banco de pruebas

Podemos utilizar full_precision para elegir instanciar el módulo con precisión completa o recortada, de ese modo utilizamos un mismo testbench para los dos módulos

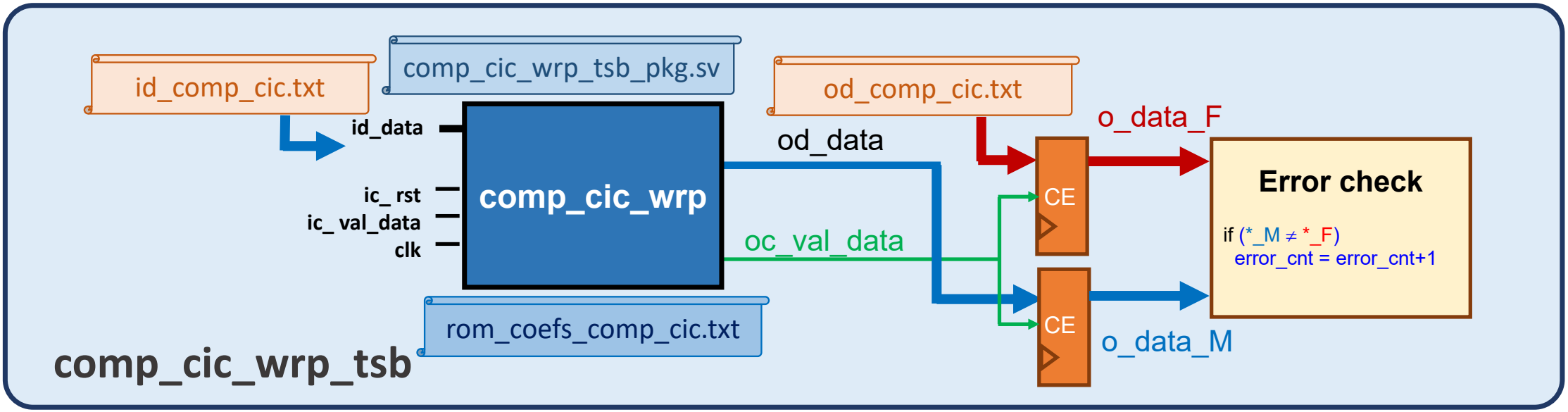
P4.1 Simulación RTL




```
############################################
#  Number of input samples          97
############################################
# TEST # 2
# Win = 16
# Wout = 18
# Wcoef = 18
# R = 2000
# Num Coefts = 17
# fclk = 96.00 MHz
# fo = 10.00 kHz
############################################
# Number of checked samples          97
# Number of errors            0
############################################
```



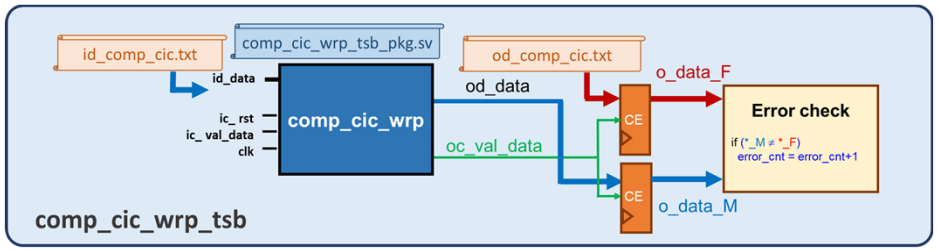
P4.1 Simulación a nivel de puertas



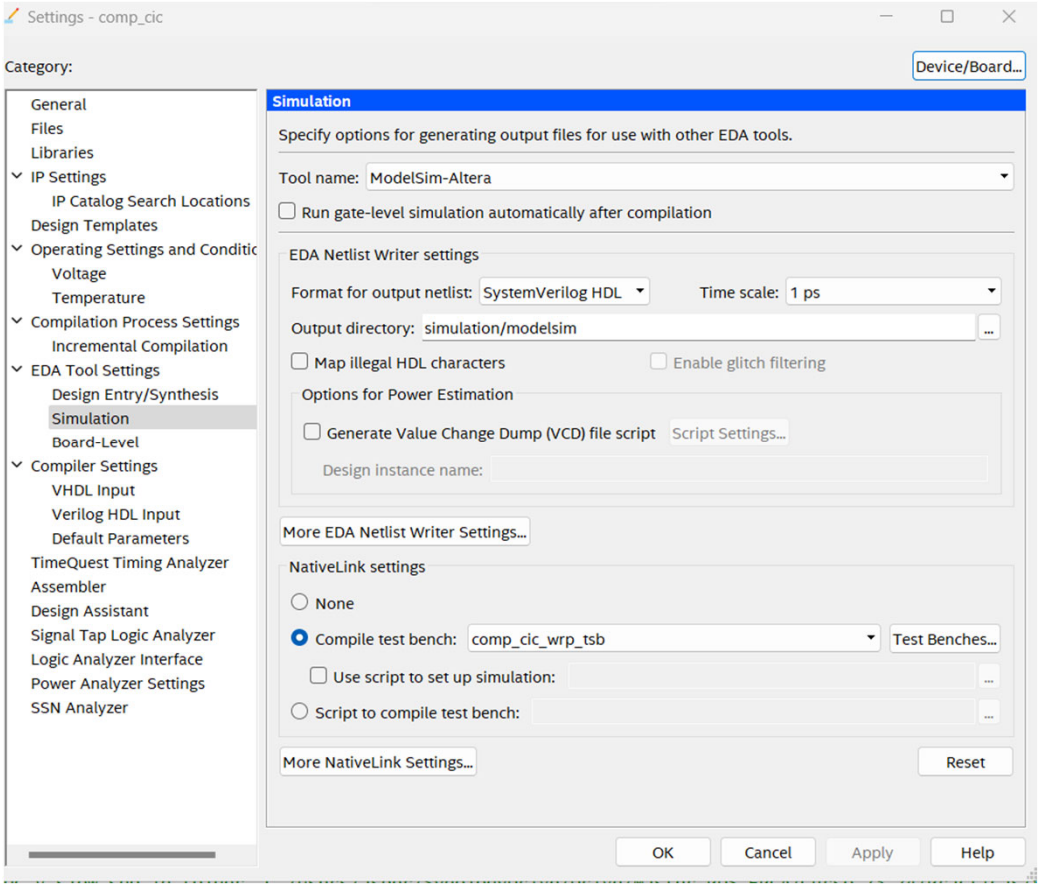
El comp_cic_wrp sin parametrizar. Incluid los valores de cuantificación del filtro con precisión recortada.
En el testbench hay que indicar la ruta en la que se encuentran los estímulos desde el directorio /quartus/simulation/modelsim:

```
data_in_file = $fopen("../../../sim/iof/id_comp_cic.txt", "r");
data_out_file = $fopen("../../../sim/iof/od_comp_cic.txt", "r");
```

P4.1 Simulación a nivel de puertas



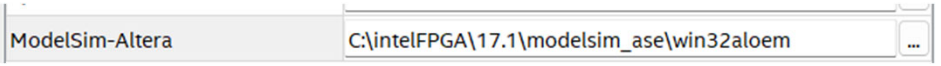
1. Compilar en Quartus el módulo comp_cic_wrp
2. Introducir el testbench Quartus > Assigments > Settings > Simulation



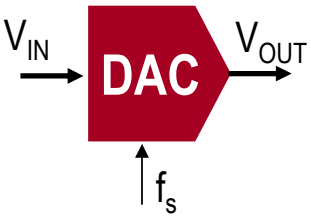
3. Simular: Quartus > Tools > Run Simulation Tools > Gate Level Simulation

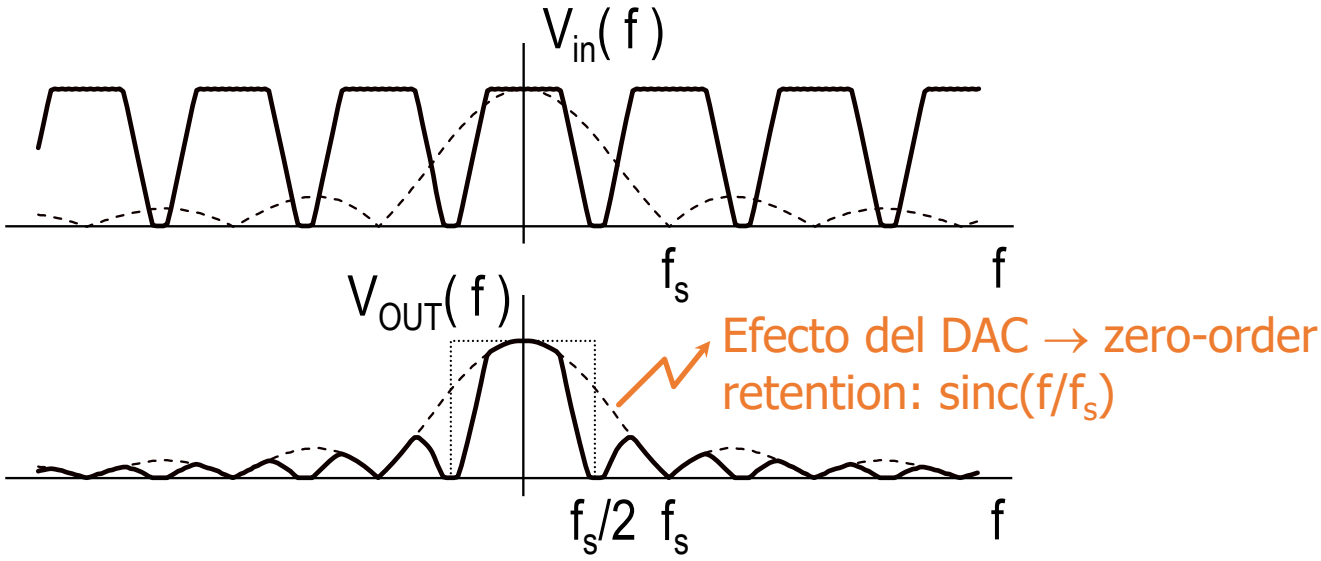
```
##########################################
#  Number of input samples          97
##########################################
# TEST # 2
# Win = 16
# Wout = 18
# Wcoef = 18
# R = 2000
# Num Coefts = 17
# fclk = 96.00 MHz
# fo = 10.00 kHz
##########################################
# Number of checked samples          97
# Number of errors          0
##########################################
```

¡¡Atención!! Si no encuentra ModelSim debéis revisar Tools > Options > General > EDA Tool Options; incluir el path del ejecutable en el apartado ModelSim-Altera



P4.2 Respuesta en frecuencia del DAC



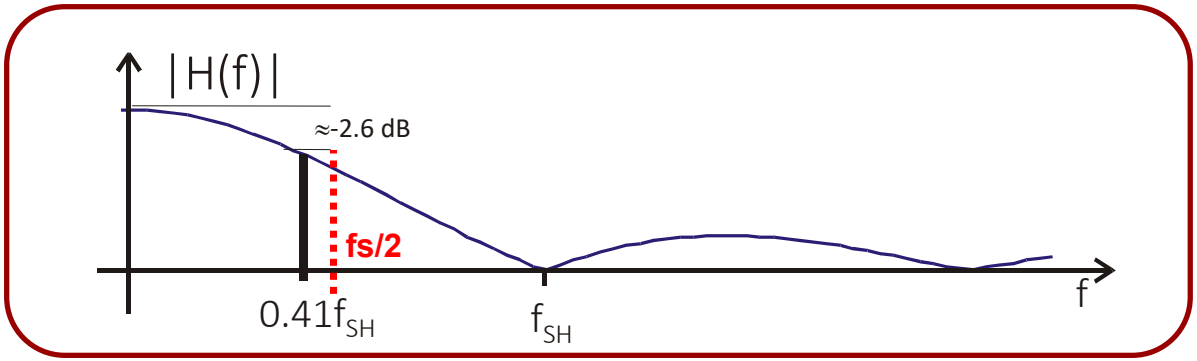


Señal modulada:

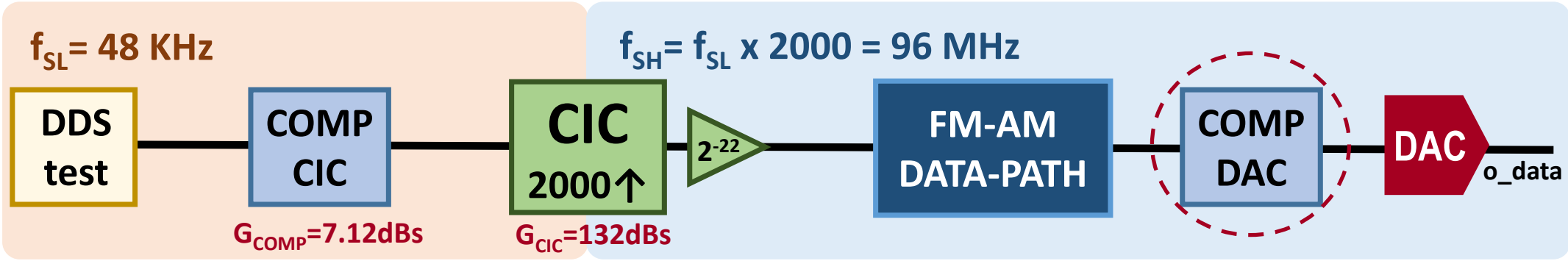
$f_c \leq 40$ MHz

Con $f_{SH} = 96$MHz

$f_{c\_max}/f_{SH} = 0.416$



P4.2 Ruta de datos: filtro compensador del DAC

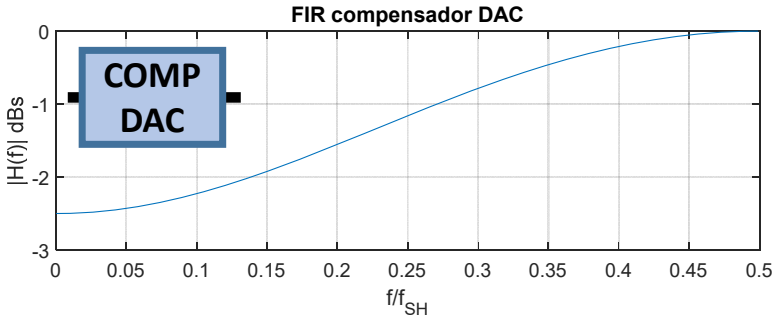


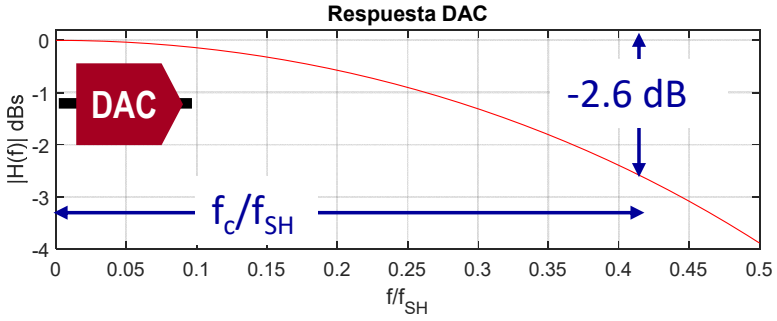
Señal modulada:

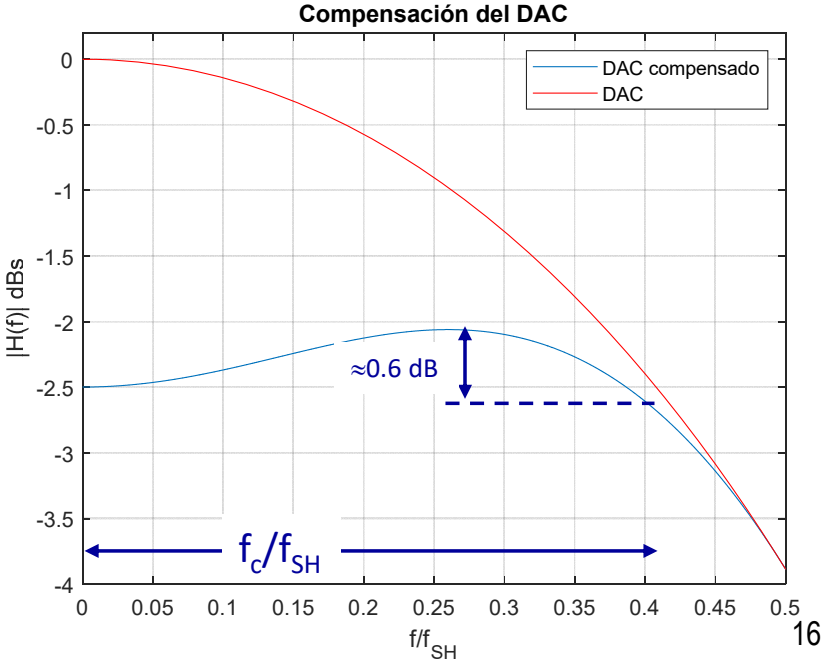
$f_c \leq 40$ MHz

Con f_{SH} = 96MHz

f_{c_max}/f_{SH} = 0.416







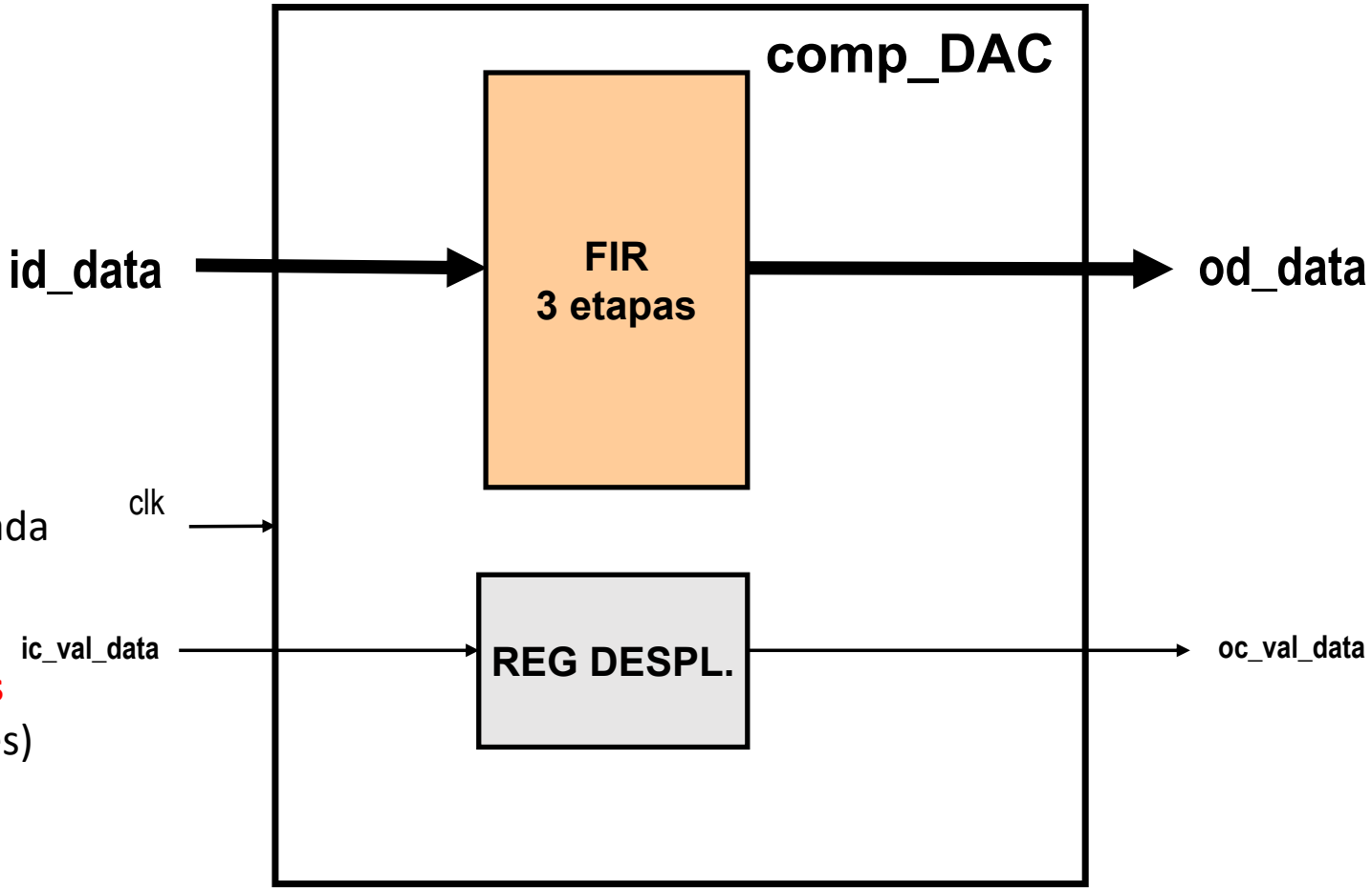
P4.2 Filtro FIR Paralelo Compensador del DAC

Especificaciones
Número de etapas: N=3
Coeficientes simétricos
h_comp_dac = [-1/16, 1-1/8, -1/16]

Formato de entrada: S[16,15]
Formato de salida: S[14,13]
FPGA Cyclone IV EP4CE115F29C7

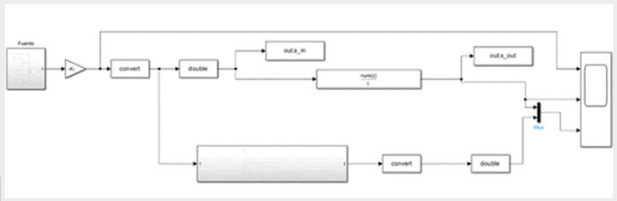
El ancho de banda de la señal de entrada será de 46 MHz

Realizar los **multiplicadores cableados** (representación CSD de los coeficientes)



P4.2 Método de verificación comp_DAC

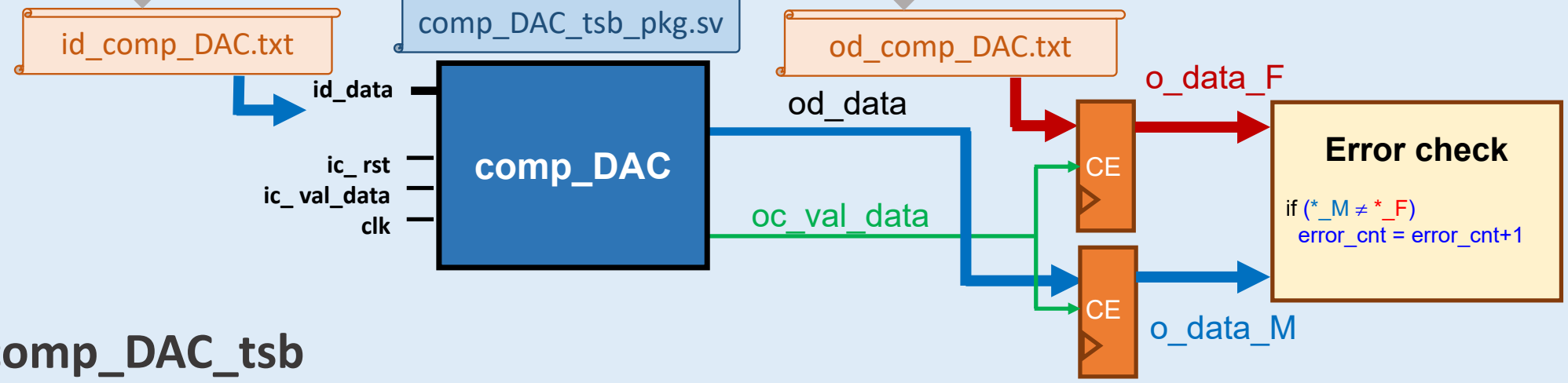
sim_comp_DAC_model.m
- Configura Win, Wout
- Lanza comp_DAC_model.slx
- Recoge datos
- Escribe ficheros de test



comp_DAC_model.slx

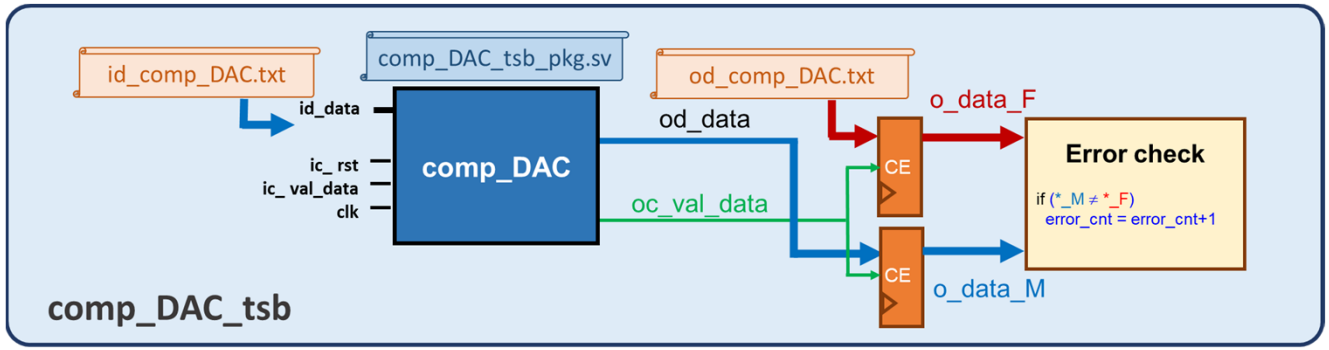
Matlab

Ficheros de P4_2
.\mat\sim_comp_DAC_model.m
.\mat\comp_DAC_model.slx
.\src\comp_DAC_pc.sv y comp_DAC.sv
.\tsb\comp_DAC_tsb.sv → Banco de prueba
.\tsb\comp_DAC_tsb_pkg.sv → Parámetros
.\sim\iof\id_comp_DAC.txt, od_comp_DAC.txt
→ Entrada y salida banco de pruebas

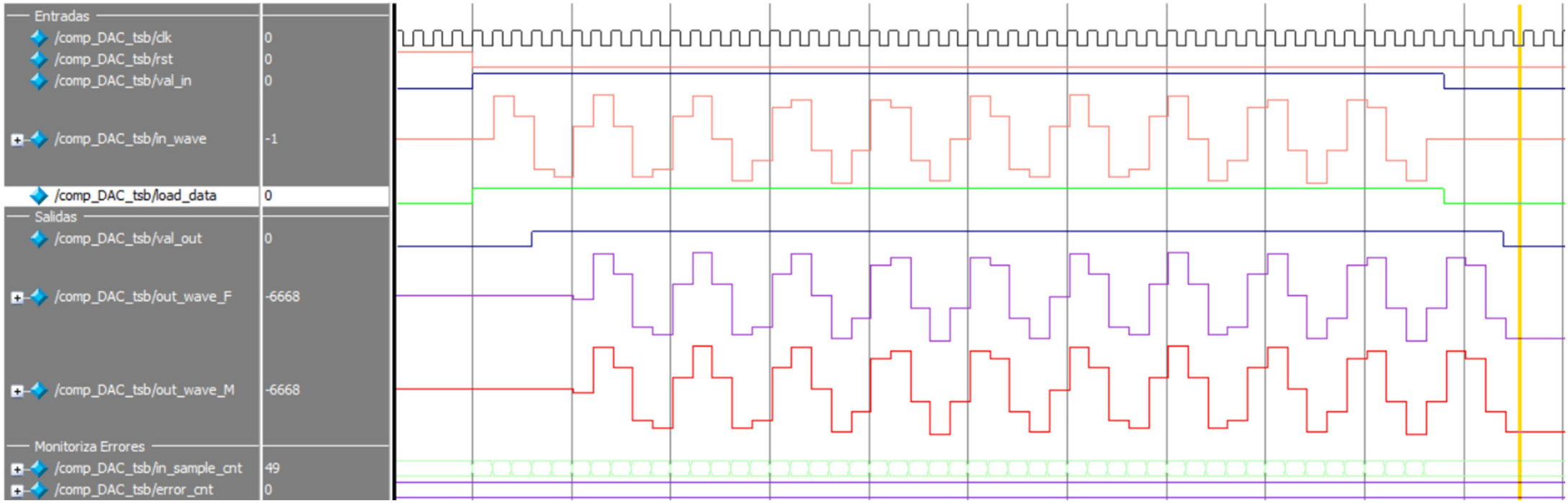


comp_DAC_tsb

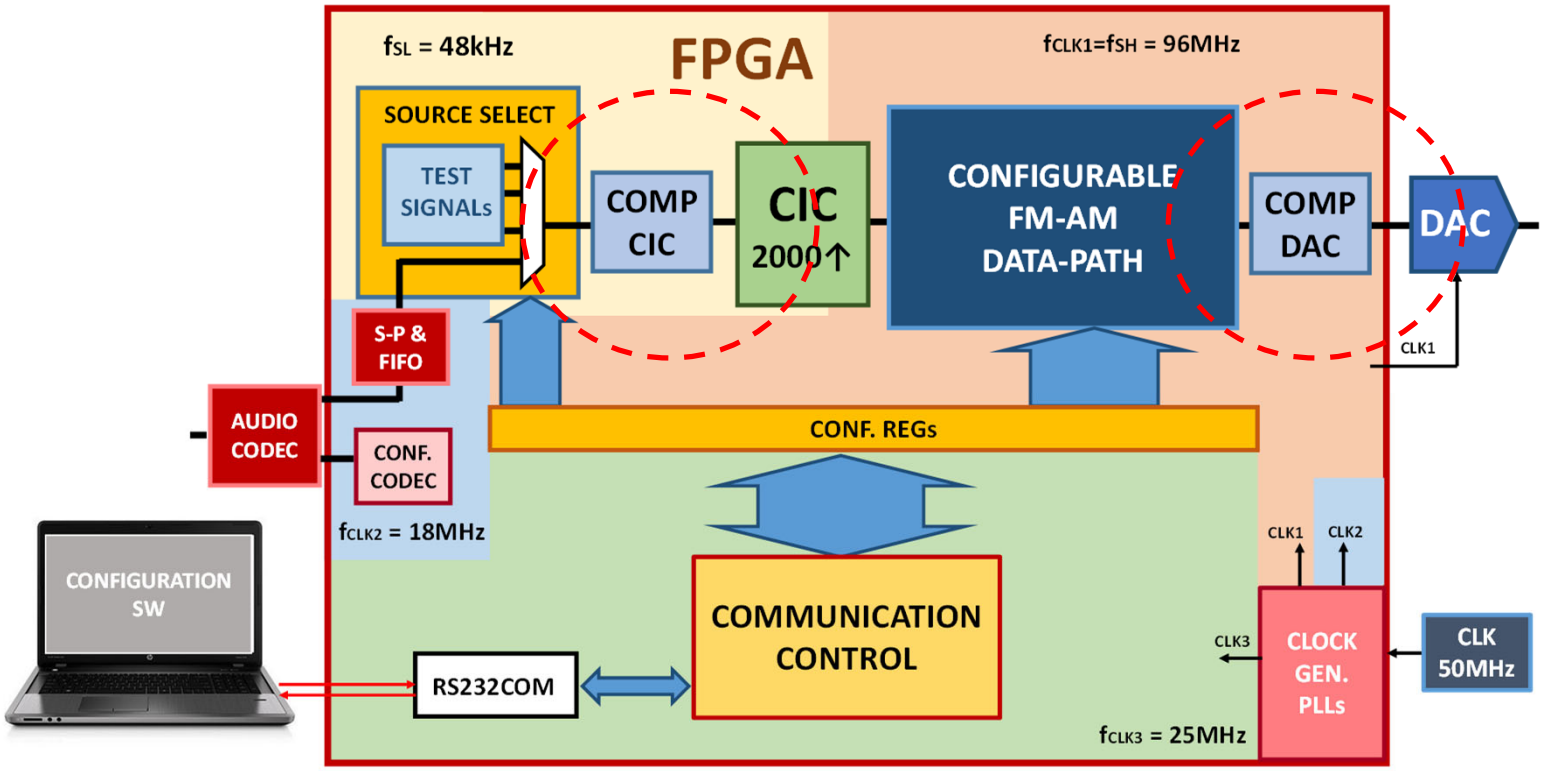
P4.2 Simulación RTL




```
##############################################
#  Number of input samples          49
##############################################
# TEST # 4
# Win = 16
# Wout = 14
##############################################
# Number of checked samples          49
# Number of errors          0
##############################################
```



P4 Modulador configurable AM/FM



P1: Sintetizador de frecuencias (DDS)
P2: Ruta de datos AM/FM configurable
P3: Filtro interpolador CIC
P4: Filtros compensadores CIC y DAC

P5: Comunicación con PC, control
P6: Completar sistema